File Handling in C

File Handling in C

File handling in C is the process of handling file operations such as creating, opening, writing data, reading data, renaming, and deleting using the C language functions. With the help of these functions, we can perform file operations to store and retrieve the data in/from the file in our program.

Need of File Handling in C

If we perform input and output operations using the C program, the data exists as long as the program is running, when the program is terminated, we cannot use that data again. **File handling** is required to work with files stored in the external memory i.e., to store and access the information to/from the computer's external memory. You can keep the data permanently using file handling.

Types of Files

A file represents a sequence of bytes. There are two types of files: **text files** and **binary files** –

- **Text file** – A text file contains data in the form of ASCII characters and is generally used to store a stream of characters. Each line in a text file ends with a new line character ("\n"), and generally has a ".txt" extension.
- **Binary file** – A binary file contains data in raw bits (0 and 1). Different application programs have different ways to represent bits and bytes and use different file formats. The image files (.png, .jpg), the executable files (.exe, .com), etc. are the examples of binary files.

FILE Pointer (File*)

While working with **file handling**, you need a **file pointer** to store the reference of the **FILE** structure returned by the fopen() function. The file pointer is required for all file-handling operations.

The fopen() function returns a pointer of the **FILE** type. **FILE** is a predefined struct type in stdio.h and contains attributes such as the file descriptor, size, and position, etc.

```
typedef struct {
    int fd;          /* File descriptor */
    unsigned char *buf;  /* Buffer */
    size_t size;     /* Size of the file */
    size_t pos;      /* Current position in the file */
} FILE;
```

Declaring a File Pointer (FILE*)

Below is the syntax to declare a file pointer –

```
FILE* file_pointer;
```

Opening (Creating) a File

A file must be opened to perform any operation. The **fopen()** function is used to create a new file or open an existing file. You need to specify the mode in which you want to open. There are various file opening modes explained below, any one of them can be used during creating/opening a file.

The **fopen()** function returns a **FILE** pointer which will be used for other operations such as reading, writing, and closing the files.

Syntax

Below is the syntax to open a file –

```
FILE *fopen(const char *filename, const char *mode);
```

Here, **filename** is the name of the file to be opened, and **mode** defines the file's opening mode.

File Opening Modes

The file access modes by default open the file in the text or ASCII mode. If you are going to handle binary files, then you will use the following access modes instead of the above-mentioned ones:

```
"rb", "wb", "ab", "rb+", "r+b", "wb+", "w+b", "ab+", "a+b"
```

There are various modes in which a file can be opened. The following are the different file opening modes –

Mode	Description
r	Opens an existing text file for reading purposes.
w	Opens a text file for writing. If it does not exist, then a new file is created. Here your program will start writing content from the beginning of the file.
a	Opens a text file for writing in appending mode. If it does not exist, then a new file is created. Here your program will start appending content in the existing file content.
r+	Opens a text file for both reading and writing.
	Opens a text file for both reading and writing. It first truncates the file to zero length if it exists, otherwise creates a file if it does not exist.
a+	Opens a text file for both reading and writing. It creates the file if it does not exist. The reading will start from the beginning but writing can only be appended.

Example of Creating a File

In the following example, we are creating a new file. The file mode to create a new file will be "**w**" (write-mode).

</> Open Compiler

```c
#include <stdio.h>

int main() {
  FILE * file;

  // Creating a file
  file = fopen("file1.txt", "w");

  // Checking whether file is
  // created or not
  if (file == NULL) {
    printf("Error in creating file");
    return 1;
  }
```

```
  printf("File created.");

  return 0;
}
```

Output

```
File created.
```

Example of Opening a File

In the following example, we are opening an existing file. The file mode to open an existing file will be "**r**" (read-only). You may also use other file opening mode options explained above.

Note: There must be a file to be opened.

```
#include <stdio.h>

int main() {
  FILE * file;

  // Opening a file
  file = fopen("file1.txt", "r");

  // Checking whether file is
  // opened or not
  if (file == NULL) {
    printf("Error in opening file");
    return 1;
  }
  printf("File opened.");

  return 0;
}
```

Output

```
File opened.
```

Closing a File

Each file must be closed after performing operations on it. The fclose() function closes an opened file.

Syntax

Below is the syntax of fclose() function –

```
int fclose(FILE *fp);
```

The fclose() function returns zero on success, or EOF if there is an error in closing the file.

The fclose() function actually flushes any data still pending in the buffer to the file, closes the file, and releases any memory used for the file. The EOF is a constant defined in the header file **stdio.h.**

Example of Closing a File

In the following example, we are closing an opened file –

</> Open Compiler

```
#include <stdio.h>

int main() {
  FILE * file;

  // Opening a file
  file = fopen("file1.txt", "w");

  // Checking whether file is
  // opened or not
  if (file == NULL) {
    printf("Error in opening file");
    return 1;
  }
  printf("File opened.");

  // Closing the file
  fclose(file);
```

```
   printf("\nFile closed.");

   return 0;
}
```

Output

```
File opened.
File closed.
```

Writing to a Text File

The following library functions are provided to write data in a file opened in writeable mode –

- **fputc()** : Writes a single character to a file.
- **fputs()**: Writes a string to a file.
- **fprintf()**: Writes a formatted string (data) to a file.

Writing Single Character to a File

The **fputc()** function is an unformatted function that writes a single character value of the argument "c" to the output stream referenced by "**fp**".

```
int fputc(int c, FILE *fp);
```

Example

In the following code, one character from a given char array is written into a file opened in the "**w**" mode:

```
#include <stdio.h>

int main() {

   FILE *fp;
   char * string = "C Programming tutorial from TutorialsPoint";
   int i;
```

```
   char ch;

   fp = fopen("file1.txt", "w");

   for (i = 0; i < strlen(string); i++) {
      ch = string[i];
      if (ch == EOF)
         break;
      fputc(ch, fp);
   }
   printf ("\n");
   fclose(fp);

   return 0;
}
```

Output

```
After executing the program, "file1.txt" will be created in the current folder and the stri
```

Writing String to a File

The **fputs()** function writes the string "s" to the output stream referenced by "fp". It returns a non-negative value on success, else EOF is returned in case of any error.

```
int fputs(const char *s, FILE *fp);
```

Example

The following program writes strings from the given two-dimensional char array to a file –

```
#include <stdio.h>

int main() {

   FILE *fp;
   char *sub[] = {"C Programming Tutorial\n", "C++ Tutorial\n", "Python
Tutorial\n", "Java Tutorial\n"};
```

```
   fp = fopen("file2.txt", "w");

   for (int i = 0; i < 4; i++) {
      fputs(sub[i], fp);
   }

   fclose(fp);

   return 0;
}
```

Output

When the program is run, a file named "file2.txt" is created in the current folder and save the following lines −

```
C Programming Tutorial
C++ Tutorial
Python Tutorial
Java Tutorial
```

Writing Formatted String to a File

The fprintf() function sends a formatted stream of data to the disk file represented by the FILE pointer.

```
int fprintf(FILE *stream, const char *format [, argument, ...])
```

Example

In the following program, we have an array of struct type called "**employee**". The structure has a string, an integer, and a float element. Using the **fprintf()** function, the data is written to a file.

```
#include <stdio.h>

struct employee {
   int age;
   float percent;
   char *name;
};
```

```
int main() {

   FILE *fp;
   struct employee emp[] = {
      {25, 65.5, "Ravi"},
      {21, 75.5, "Roshan"},
      {24, 60.5, "Reena"}
   };

   char *string;
   fp = fopen("file3.txt", "w");

   for (int i = 0; i < 3; i++) {
      fprintf(fp, "%d %f %s\n", emp[i].age, emp[i].percent, emp[i].name);
   }
   fclose(fp);

   return 0;
}
```

Output

```
When the above program is executed, a text file is created with the name "file3.txt" tha
```

Reading from a Text File

The following library functions are provided to read data from a file that is opened in read mode −

- **fgetc()**: Reads a single character from a file.
- **fgets()**: Reads a string from a file.
- **fscanf()**: Reads a formatted string from a file.

Reading Single Character from a File

The **fgetc()** function reads a character from the input file referenced by "**fp**". The return value is the character read, or in case of any error, it returns EOF.

```
int fgetc(FILE * fp);
```

Example

The following example reads the given file in a character by character manner till it reaches the end of file.

</> Open Compiler

```
#include <stdio.h>

int main(){

   FILE *fp ;
   char ch ;
   fp = fopen ("file1.txt", "r");

   while(1) {
      ch = fgetc (fp);
      if (ch == EOF)
         break;
      printf ("%c", ch);
   }
   printf ("\n");
   fclose (fp);
}
```

Output

Run the code and check its output –

```
C Programming tutorial from TutorialsPoint
```

Reading String from a File

The **fgets()** function reads up to "n − 1" characters from the input stream referenced by "fp". It copies the read string into the buffer "**buf**", appending a null character to terminate the string.

Example

This following program reads each line in the given file till the end of the file is detected −

```
# include <stdio.h>

int main() {

   FILE *fp;
   char *string;
   fp = fopen ("file2.txt", "r");

   while (!feof(fp)) {
      fgets(string, 256, fp);
      printf ("%s", string) ;
   }
   fclose (fp);
}
```

Output

Run the code and check its output −

```
C Programming Tutorial
C++ Tutorial
Python Tutorial
Java Tutorial
```

Reading Formatted String from a File

The fscanf() function in C programming language is used to read formatted input from a file.

```
int fscanf(FILE *stream, const char *format, ...)
```

Example

In the following program, we use the **fscanf()** function to read the formatted data in different types of variables. Usual format specifiers are used to indicate the field types (%d, %f, %s, etc.)

```c
#include <stdio.h>

int main() {

   FILE *fp;
   char *s;
   int i, a;
   float p;

   fp = fopen ("file3.txt", "r");

   if (fp == NULL) {
      puts ("Cannot open file"); return 0;
   }

   while (fscanf(fp, "%d %f %s", &a, &p, s) != EOF)
   printf ("Name: %s Age: %d Percent: %f\n", s, a, p);
   fclose(fp);

   return 0;
}
```

Output

When the above program is executed, it opens the text file "file3.txt" and prints its contents on the screen. After running the code, you will get an output like this –

```
Name: Ravi Age: 25 Percent: 65.500000
Name: Roshan Age: 21 Percent: 75.500000
Name: Reena Age: 24 Percent: 60.500000
```

File Handing Binary Read and Write Functions

The read/write operations are done in a binary form in the case of a binary file. You need to include the character "**b**" in the access mode ("**wb**" for writing a binary file, "**rb**" for reading a binary file).

There are two functions that can be used for binary input and output: the **fread()** function and the **fwrite()** function. Both of these functions should be used to read or write blocks of memories, usually arrays or structures.

Writing to Binary File

The **fwrite()** function writes a specified chunk of bytes from a buffer to a file opened in binary write mode. Here is the prototype to use this function:

```
fwrite(*buffer, size, no, FILE);
```

Example

In the following program, an array of a struct type called "**employee**" has been declared. We use the **fwrite()** function to write one block of byte, equivalent to the size of one **employee** data, in a file that is opened in "**wb**" mode.

```
#include <stdio.h>

struct employee {
   int age;
   float percent;
   char name[10];
};

int main() {

   FILE *fp;
   struct employee e[] = {
      {25, 65.5, "Ravi"},
      {21, 75.5, "Roshan"},
      {24, 60.5, "Reena"}
   };

   char *string;

   fp = fopen("file4.dat", "wb");
   for (int i = 0; i < 3; i++) {
      fwrite(&e[i], sizeof (struct employee), 1, fp);
   }

   fclose(fp);

   return 0;
}
```

Output

When the above program is run, the given file will be created in the current folder. It wi

Reading from Binary File

The **fread()** function reads a specified chunk of bytes from a file opened in binary read mode to a buffer of the specified size. Here is the prototype to use this function:

```
fread(*buffer, size, no, FILE);
```

Example

In the following program, an array of a struct type called "**employee**" has been declared. We use the **fread()** function to read one block of byte, equivalent to the size of one **employee** data, in a file that is opened in "**rb**" mode.

```
#include <stdio.h>

struct employee {
   int age;
   float percent;
   char name[10];
};

int main() {

   FILE *fp;
   struct employee e;

   fp = fopen ("file4.dat", "rb");

   if (fp == NULL) {
      puts ("Cannot open file");
      return 0;
   }

   while (fread (&e, sizeof (struct employee), 1, fp) == 1)
   printf ("Name: %s Age: %d Percent: %f\n", e.name, e.age, e.percent);

   fclose(fp);
```

```
   return 0;
}
```

Output

When the above program is executed, it opens the file "file4.dat" and prints its contents on the screen. After running the code, you will get an output like this –

```
Name: Ravi Age: 25 Percent: 65.500000
Name: Roshan Age: 21 Percent: 75.500000
Name: Reena Age: 24 Percent: 60.500000
```

Renaming a File

The rename() function is used to rename an existing file from an old file name to a new file name.

Syntax

Below is the syntax to rename a file –

```
int rename(const char *old_filename, const char *new_filename)
```

Example

```
#include <stdio.h>

int main() {
  // old and new file names
  char * file_name1 = "file1.txt";
  char * file_name2 = "file2.txt";

  // Renaming old file name to new one
  if (rename(file_name1, file_name2) == 0) {
    printf("File renamed successfully.\n");
  } else {
    perror("There is an error.");
  }
```

```
   return 0;
}
```

Output

If there is a file (file1.txt) available, the following will be the output –

```
Name: Ravi Age: 25 Percent: 65.500000
Name: Roshan Age: 21 Percent: 75.500000
Name: Reena Age: 24 Percent: 60.500000
```